

Machine-Verifiable Item Models for Geometry Diagram Generation

Anonymous submission

Abstract

Language models can draw diagrams that look right but are geometrically wrong. We have them specify constructions, not coordinates; a symbolic engine computes exact figures and checks geometric predicates, enabling automatic item generation with computed answer keys. Structured generation improves pass rates; human validation reveals limits in checker completeness.

1 Introduction

Educational testing runs on a steady supply of fresh items, and writing each one by hand—together with the key used to score it—is slow, expensive, and a bottleneck every testing program feels. For decades the field’s answer has been *automatic item generation* (AIG): an author writes a single template, or *item model*, and a computer mints many parallel items from it, each carrying its own answer key (Bejar, 2002; Gierl and Haladyna, 2013). AIG has worked best for items built from text; the ones it has struggled to reach are those whose response is a *picture*. In K–12 mathematics, the picture that matters most is the geometry diagram.

Producing a correct one is harder than it looks. Take a simple request:

“Draw an acute triangle ABC with $\angle A = 60^\circ$ and $\angle B = 70^\circ$, then draw the altitude from C , meeting AB at H .”

To be correct, the drawing has to get several things *exactly* right: angle A is 60° , the point H lies on the segment AB (not past its end), and the altitude meets AB at a true right angle. If a model produces a drawing by guessing coordinates, these facts hold only by luck. And when they fail, they fail silently: the result still looks like a triangle with a line through it, even if that right angle is really 88° or the foot of the altitude slipped just past B .

The same exactness makes such diagrams hard to score. Almost every geometry benchmark runs the task backwards—it shows the model a finished diagram and asks a question about it (Lu et al., 2021, 2024)—so none checks whether a model can *produce* a correct figure in the first place. The benchmarks that do score generated pictures were built for everyday images and rely on an AI “judge” that looks at the output (Huang et al., 2025; Wang et al., 2026). A judge can say a shape *looks like* a right triangle, but it cannot certify that an angle is exactly 90° . For geometry diagrams, then, neither making the item nor scoring it comes for free.

Our idea is to stop guessing and stop looking. We have the model describe the construction in words—“ H is the foot of the perpendicular from C to AB ”—and hand that description to a geometry engine that computes the exact coordinates. The model chooses *what* to build; the engine works out *where* every point goes. That single move answers both problems at once:

- **Better drawings.** The hard arithmetic is done by a solver, so a right angle comes out exactly right instead of merely close.
- **Automatic scoring.** Because the finished figure is now exact numbers, we can check each requirement directly—“is this angle 90° ?”—rather than asking a judge what the picture looks like.

That is this paper’s contribution: a *machine-verifiable item-model framework* for geometry diagrams. From a single construction, one item model emits both the prompt and a *computed* answer key—a machine-checkable list of geometric requirements—and the figure produced in response is verified against that key. We make two empirical claims and are deliberate about their strength: structured construction improves how

often models get the geometry right (Section 4), but human validation shows the automatic score is best read as an *upper bound* on correctness, not a validated high-stakes score (Section 3). A fuller, per-cell leaderboard and cost analysis appear in a companion paper; here the benchmark is evidence for these two claims, not the contribution itself.

The rest of the paper walks through how the model describes a diagram (Section 2), how scoring follows from that same description (Section 3), what we found running it on six large models (Section 4), and why this is useful for educational measurement (Section 5).

2 How the model draws a diagram

A request becomes a picture in three steps, shown in Figure 1. The model writes a short, structured description of the construction; a geometry engine turns that description into exact coordinates; and a renderer turns the coordinates into a drawing. The important idea is the division of labour: **the model decides what to build, and the engine decides where the points go**. The model never writes a coordinate.

Step 1: a description, not coordinates. The model fills in a simple, typed form—we call it the intermediate representation, or IR. The form offers 31 kinds of building block: points, segments, lines, circles, ellipses, polygons, and intersections. Each block names a *relationship* rather than a position. A point can be “the midpoint of A and B ,” “where these two lines cross,” or “the foot of the perpendicular from C .” In our example the model writes “ H is where line AB meets the perpendicular to AB through C ”—never “ $H = (3.07, 0)$.” Because the form is typed, obvious mistakes (a missing point, the wrong number of arguments) are caught right away, before any geometry is computed.

Step 2: the engine solves for the points. A geometry engine—we use SymPy, a symbolic-mathematics library—reads the description and works out the coordinates, handling the building blocks in dependency order. Symbolic methods have made headlines for solving olympiad geometry (Trinh et al., 2024); here the engine does a humbler job, working out where the points of a described figure go. When a step is ambiguous (two circles cross at two points), the description simply says which one to keep (“the higher one”). The

engine, not the model, produces every coordinate. This is what makes the right angle in our example exactly 90° rather than approximately so.

Step 3: render. From the exact coordinates the system writes standard drawing code and compiles it to an image. Fiddly details an author should not have to manage—such as clipping a line at the edge of the canvas—are handled automatically.

Two ways to drive it. The model can fill in the form directly, or it can use a short “recipe” language that names a construction—`perp_bisector(A, B)`—which then expands into the same form. Either way, the description can carry its own requirements, tested *before* the figure is drawn, so the model can catch a mistake and retry. We compare both against a plain baseline in which the model just writes drawing code by hand, with no engine and no checks; that baseline is how we measure what the structured approach is worth (Section 4). Having a model write drawing code directly is itself an active line of work (Belouadi et al., 2024a,b); that work asks how closely the output resembles a target picture, whereas we ask whether its geometry checks out.

3 Automatic scoring from shared construction logic

Because the finished diagram is just exact coordinates, many of a prompt’s requirements can be scored directly, as explicit geometric predicates. “Is the angle at H a right angle?” is settled by a one-line calculation on the coordinates—in a fraction of a millisecond—not by asking a judge whether the picture looks square. This makes scoring inspectable and inexpensive, but not complete: the checklist’s validity still depends on whether it captures everything the prompt asks for and the visual conventions a reader expects—limits we examine below.

The question and its answer key are written together. Here is the move that makes this practical: each item and its answer key come from the *same* short program (Figure 2). One template produces both the wording shown to the model and the checklist used to score it. Because the two are emitted together, they are far less likely to drift apart through manual editing—though template-level errors remain possible: an author can still omit a predicate, encode the wrong one, or write wording whose intended constraint the checklist

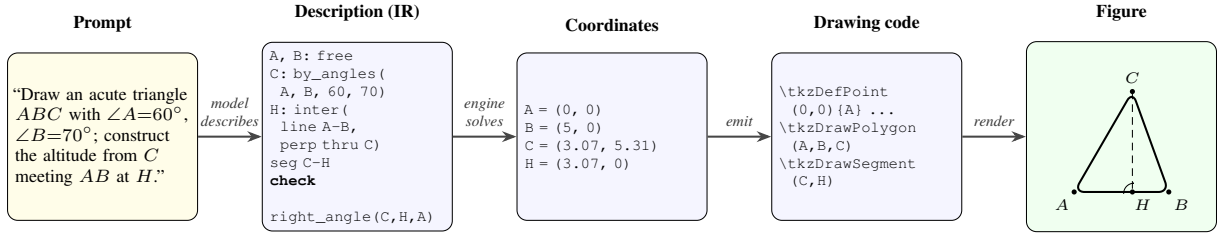


Figure 1: One request, start to finish. The model writes a description of the construction; the geometry engine solves it for exact coordinates; the system emits drawing code and renders it. Because the model writes relationships (“ H is the foot of the perpendicular from C ”) rather than numbers, the geometry is computed rather than guessed—and those same coordinates are what we score against in Section 3.

Check	What it confirms
right_angle	an angle is a right angle
midpoint	a point is halfway along a segment
collinear / parallel	points lie on a line / lines never meet
point_on_segment	a point lies between two end-points
equal_lengths	two segments have equal length
angle_equal	an angle equals a stated value

Table 1: A sample of the 15 checks in the scoring vocabulary. Each is decided exactly on the computed coordinates; an item’s list of checks is, in plain terms, what a passing answer must satisfy.

does not fully capture. Re-running the template with a new random seed still mints a fresh, parallel item, and the checklist is never shown to the model while it works.

A small, readable checklist. Every requirement is written in a small, fixed vocabulary of checks (Table 1): is this a right angle, is this the midpoint, are these lines parallel, does this point lie on that segment, and so on. There are 15 such checks in the current version, and each is decided exactly on the coordinates (within a tiny rounding tolerance). A diagram passes only if every check on its list passes. The list is, in plain terms, the definition of what the item is testing.

Does the automatic score match a human’s?

An automatic score is only worth having if it agrees with a careful human grader, so we checked. After fixing a few blind spots found on pilot data, we ran a pre-registered study. We set the bar in advance: the automatic score had to match the human verdict at Cohen’s $\kappa \geq 0.70$ (on a scale where 1.0 is perfect agreement and 0 is chance). Two graduate-level graders then scored 200 diagrams by hand, without seeing the automatic verdict.

We did not clear that bar. Agreement was only moderate—Cohen’s $\kappa = 0.46$, well short of the pre-registered 0.70—and we treat this as a central limitation, not a footnote. The disagreements do run *one way*: in 36 of the 200 cases the automatic scorer passed a diagram the graders failed, and in *zero* cases was it too harsh. That one-sidedness is good for fairness—no model is penalised unjustly—but it is the *dangerous* direction once a score is read as a certificate of correctness, because every disagreement is a false positive. We therefore state it plainly: **the model pass rates we report in Section 4 are optimistic ceilings**—upper bounds on how often the geometry is truly right, not the rate itself. The errors do concentrate in one place—items with an added construction (like our altitude), where the checklist confirms the construction but does not separately re-check the prompt’s exact angles—and the two graders agreed with *each other* at the 0.70 we had asked the scorer to reach ($\kappa = 0.70$). So the gap is in the checklist, not the human ratings; the fix, adding the missing numeric checks, is straightforward and underway, and until it lands the reported rates should be read as ceilings.

What the checks cannot see. That particular gap closes with more checks. A deeper one does not. The checks read the geometry—the coordinates—and nothing else; they never look at how the figure is drawn. A diagram can satisfy every requirement and still mislead a reader: a right-angle mark placed on the outside of the corner, an angle labelled on the wrong side, or a figure that passes every check yet is laid out so awkwardly that no reader could follow it. This is the mirror image of the AI judge from Section 1. A judge sees how a picture looks but cannot check its geometry; our checks test that geometry but cannot see how it looks. The two are complementary:

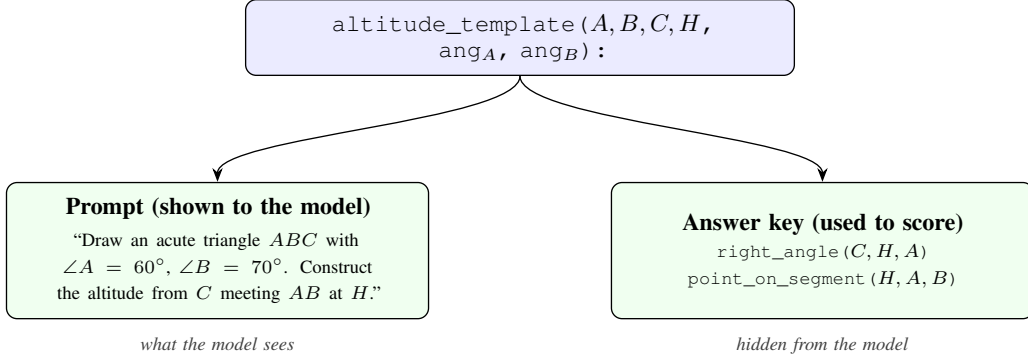


Figure 2: One template produces both the question and its answer key, so the two cannot drift apart.

the checks speak to whether a diagram is *correct*, while whether it is *clear* is still best judged by eye.

4 Putting it to the test

To show the approach works beyond a single example, we evaluate on GEOGENBENCH, 801 prompts, all scored automatically by the checker of Section 3. Six hundred come from the templates described above; another 201 are drawn from a real K–12 mathematics curriculum (Carnegie Learning, Inc. and Texas Education Agency, 2024). We will release the prompt set, templates, checker vocabulary, and rendered outputs upon publication. We ran six large models from three companies, each in two modes: writing drawing code by hand, and using the structured description. Throughout, a *pass* means passing that automatic checker; since the checker is one-directionally lenient (Section 3), the rates below are optimistic ceilings on true geometric correctness, not exact-correctness rates.

How we ran it. The by-hand baseline asks the model to write TikZ drawing code directly; the structured mode fills in the typed form of Section 2. Both modes run at temperature 1.0, with no explicit output-token limit and the *same* budget of up to three attempts—what differs is the retry trigger. The baseline retries only when its code fails to render; the structured mode also retries when its description fails to compile or the figure breaks a check it declared. Those declared checks are the model’s own, though: the held-out scoring predicates the item template generates—the ones we actually grade against—are never fed back to it. The structured advantage is therefore not more attempts but more feedback—it lets a model catch its own geometric mistakes, which

Model	By hand	Structured	\$/item
GPT-5.5	91.9%	94.1%	0.075
Claude Sonnet 4.6	81.2%	93.4%	0.088
Claude Opus 4.7	71.6%	93.3%	0.53
Claude Haiku 4.5	70.5%	75.3%	0.032
Gemini 2.5 Pro	58.4%	83.1%	0.050
Gemini 2.5 Flash	35.9%	blocked	—

Table 2: Share of the 600 templated prompts *passing the current automatic checker*—drawing by hand versus using the structured description—with the average cost of one structured generation in US dollars. The checker is one-directionally lenient (Section 3), so these rates are optimistic ceilings on true correctness. Every model improves, the weakest most; the three strongest converge near 93–94% despite a sevenfold cost spread. Gemini 2.5 Flash could not run the structured form (rejected as too complex; see text).

hand-written code cannot, and those extra retries show up in its per-item cost. Each requirement is decided on the computed coordinates within an absolute tolerance of $\tau = 5 \times 10^{-3}$ (coordinates normalised to $[-10, 10]$). The 600 templated prompts divide evenly across three difficulty tiers (200 each), generated from 30 construction templates—triangles, quadrilaterals, circles, segments, and lines—with 3.2 requirements apiece on average; the 201 curriculum prompts were written from the topic scaffold of a public K–12 geometry curriculum (drafted with a language model, answer keys written by hand) and carry more, 6.0 on average. Cost per item is the prompt’s token count, summed across any retries and priced at each vendor’s published rates, then averaged over every prompt—failures included. Failures are dominated by wrong geometry—a requirement that simply does not hold—rather than broken drawing code, and the structured mode’s remaining failures skew toward timeouts.

The structured approach helped *every* model (Table 2)—and helped the weakest ones the most, raising pass rates by anywhere from 2 to 25 percentage points. In other words, handing a model the geometry engine matters most when the model is poor at drawing by hand. More striking still: once they used the structured approach, the three strongest models landed within a single point of one another, even though they differ sevenfold in price. On this task, the method does more of the work than the model does.

We also hit one practical snag worth flagging for anyone building something similar: Gemini 2.5 Flash refused our structured form outright for being too complex, so the structured mode could not run there at all. Additional per-template results, cost breakdowns, and failure analyses appear in the companion paper. Here they make a single point: structured construction helps at scale, across every model we tried.

5 Why this matters for measurement

Seen through the lens of educational measurement, this is a way to write test items and score them automatically—at the same time.

Each template is an item generator. Fixing every parameter but one and sweeping that parameter—an angle through 30° , 40° , 50° , say—mints a family of structurally parallel questions, and the checker confirms each variant without anyone looking (Figure 3). An author thus has an unlimited, refreshable supply from a single construction. These variants are parallel by construction; whether they are also of equal empirical difficulty—true psychometric isomorphs—is a calibration question we have not yet tested. And because the answer key is produced *with* each item rather than written afterward, the most common source of scoring error—a hand-written key that has drifted from its question—is largely designed out. In the vocabulary of the field the template is an *item model* (Gierl and Lai, 2012), now one that emits its own key. That move also shifts where quality control happens, in the way the item-generation literature recommends: rather than vetting every generated item, one reviews the construction logic that produces them all (Gierl and Lai, 2016).

The diagram a student (or a model) produces is a constructed response, and the same description that built the item can score it. Here a measure-

ment audience should weigh two claims separately. The *generation* claim is the robust one: because the key is computed with the item rather than written separately, question and key are far less likely to drift apart, and that key is a short, readable list of exactly what must be true (Table 1)—open to inspection rather than buried inside a holistic rubric. The *scoring* claim has to be earned empirically, and our validity study (Section 3) is candid that it is not yet earned: the auto-score agrees with human graders only moderately and errs in a single direction, so for now it is best read as a reliable upper bound on correctness—fit for benchmarking and for flagging likely-wrong work, but not yet for standalone high-stakes scoring. Closing that gap means tightening the soft keys that Section 3 pinpoints, which is in progress.

Scope. The version described here covers plane geometry. The same machinery should extend to coordinate graphs and 3D solids by adding new kinds of building block, though we have not built those yet. And where a topic has no natural template, an author can still write items and keys by hand and score them with the very same checker. The core idea stays simple: when a diagram is built from an exact description, that same description tells you whether the diagram is right.

References

- Isaac I. Bejar. 2002. Generative testing: From conception to implementation. In Sidney H. Irvine and Patrick C. Kyllonen, editors, *Item Generation for Test Development*, pages 199–217. Lawrence Erlbaum Associates.
- Jonas Belouadi, Anne Lauscher, and Steffen Eger. 2024a. AutomaTikZ: Text-guided synthesis of scientific vector graphics with TikZ. In *Proc. Int. Conf. on Learning Representations (ICLR)*.
- Jonas Belouadi, Simone Paolo Ponzetto, and Steffen Eger. 2024b. DeTikZify: Synthesizing graphics programs for scientific figures and sketches with TikZ. In *Proc. Conf. on Neural Information Processing Systems (NeurIPS)*.
- Carnegie Learning, Inc. and Texas Education Agency. 2024. Bluebonnet learning mathematics: Teacher’s edition (grades 6–8 and geometry). Curriculum materials adopted under the Texas State Board of Education Instructional Materials and Technology Allotment. Accessed via the Texas Education Agency Bluebonnet Learning programme; <https://learn.texas.gov/>.

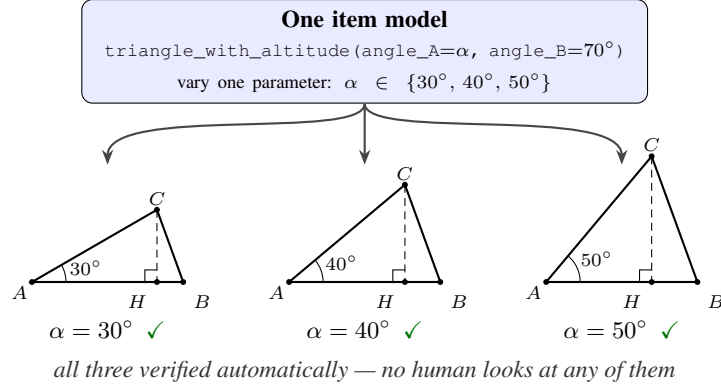


Figure 3: From one item model to a family of verified items—automatic item generation with the answer key built in. Holding every parameter fixed but the angle at A and sweeping it through $30^\circ, 40^\circ, 50^\circ$ produces parallel variants; for each, the checker of Section 3 confirms the altitude from C meets AB at a right angle ($\checkmark$), with no human review. One model yields an unlimited, refreshable supply of items, each carrying its own key.

Mark J. Gierl and Thomas M. Haladyna, editors. 2013. *Automatic Item Generation: Theory and Practice*. Routledge.

Mark J. Gierl and Hollis Lai. 2012. The role of item models in automatic item generation. *International Journal of Testing*, 12(3):273–298.

Mark J. Gierl and Hollis Lai. 2016. [A process for reviewing and evaluating generated test items](#). *Educational Measurement: Issues and Practice*, 35(4):6–20.

Kaiyi Huang, Chengqi Duan, Kaiyue Sun, Enze Xie, Zhenguo Li, and Xihui Liu. 2025. T2I-CompBench++: An enhanced and comprehensive benchmark for compositional text-to-image generation. *IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI)*.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2024. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

Pan Lu, Ran Gong, Shibiao Jiang, Liang Qiu, Siyuan Huang, Xiaodan Liang, and Song-Chun Zhu. 2021. Inter-GPS: Interpretable geometry problem solving with formal language and symbolic reasoning. In *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*.

Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2024. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482.

Zhaokai Wang, Penghao Yin, Xiangyu Zhao, Changyao Tian, Yu Qiao, Wenhai Wang, Jifeng Dai, and Gen Luo. 2026. GenExam: A multidisciplinary text-to-image exam. In *Proc. Int. Conf. on Machine Learning (ICML)*.